

Guia Prático de

TraceQL

Tempo Query Language — Grafana Tempo

Este guia foi criado para ajudar qualquer usuário — mesmo sem experiência técnica — a entender e utilizar o TraceQL para consultar traces distribuídos no Grafana Tempo. Você aprenderá a rastrear requisições ponta a ponta, identificar gargalos de latência e correlacionar traces com logs e métricas.

Conteúdo deste Guia

01	O que é TraceQL e conceitos fundamentais de tracing
02	Estrutura de uma query — Spans, atributos e filtros
03	Operadores, tipos de dados e condições avançadas
04	Agregações, métricas de traces e exemplos práticos

01 — O que é TraceQL?

TraceQL é a linguagem de consulta do **Grafana Tempo**, ferramenta de rastreamento distribuído (distributed tracing). Enquanto o PromQL analisa métricas e o LogQL analisa logs, o TraceQL permite **rastrear o caminho completo de uma requisição** por todos os microserviços que ela passou — identificando onde ocorreu lentidão ou falha.

Conceitos fundamentais:

Conceito	O que é	Analogia prática
Trace	Jornada completa de uma requisição	O percurso total de um pedido no app
Span	Etapa individual dentro do trace	Cada serviço que o pedido passou
Root Span	Span inicial que originou o trace	O clique do usuário que iniciou tudo
Atributo	Metadado de um span (chave=valor)	http.method='POST', status=200
Duration	Tempo de execução de um span	Quanto tempo o banco demorou

02 — Estrutura de uma Query TraceQL

Uma query TraceQL é formada por **span sets** dentro de chaves **{ }**. Cada bloco filtra spans por atributos. O resultado retorna os traces que contêm ao menos um span que satisfaz todas as condições:

■ Anatomia de uma query TraceQL

```
{ span.http.method = "POST" && span.http.status_code >= 500 }

# Filtro de span
# span. → prefixo para atributos do span
# && → operador AND (ambas as condições devem ser verdade)
# Retorna: todos os traces com spans POST que retornaram erro 5xx
```

Prefixos de atributos disponíveis:

Prefixo	Escopo	Exemplo
span.	Atributo do span individual	<code>span.http.url</code>
resource.	Atributo do serviço/processo	<code>resource.service.name</code>
trace.	Atributo do trace inteiro	<code>trace.duration</code>
.attr	Busca em span e resource	<code>.http.method</code>
duration	Duração do span (sem prefixo)	<code>duration > 500ms</code>
status	Status do span: ok/error/unset	<code>status = error</code>
name	Nome da operação do span	<code>name = "checkout"</code>

■ **Dica:** Use `resource.service.name` para filtrar por serviço específico — é o atributo mais comum e está presente em praticamente todos os traces instrumentados com OpenTelemetry.

03 — Operadores e Tipos de Dados

O TraceQL suporta diferentes tipos de dados nos atributos. É importante usar o tipo certo na comparação para obter resultados corretos:

Operadores de comparação:

Operador	Uso	Exemplo
=	Igual	span.http.method = "GET"
!=	Diferente	status != ok
>	Maior que	duration > 1s
>=	Maior ou igual	span.http.status_code >= 400
<	Menor que	duration < 200ms
~=	Regex	resource.service.name ~= "pay.*"
!~	Não regex	name !~ "health.*"

Unidades de duração e tipos:

Tipo	Formato	Exemplos
Duração	número + unidade	200ms / 1.5s / 2m / 1h
String	entre aspas duplas	"POST" / "pagamentos" / "br-east"
Número	inteiro ou decimal	200 / 500 / 1024
Boolean	true / false	true / false
Status	palavras-chave	ok / error / unset
Nil	ausência de valor	span.http.url != nil

Condições avançadas — múltiplos spans no mesmo trace

O TraceQL permite buscar traces onde **diferentes spans** satisfazem condições distintas, usando o operador >> (descendente) para relacionar spans pai e filho:

■ Spans encadeados — pai chama filho com erro

```
{ resource.service.name = "api-gateway" }
>> { resource.service.name = "db-service" && status = error }

# Encontra traces onde o api-gateway chamou o db-service
# E o db-service retornou erro – rastreia a origem do problema
```

■ Operadores lógicos && e || dentro do span

```
{ span.http.method = "POST"
&& span.http.status_code >= 500
&& duration > 2s }

# POST com erro 5xx E que demorou mais de 2 segundos
# Identifica requisições lentas E com falha ao mesmo tempo
```

■ **Dica:** Combine **status = error** com **duration > Xs** para encontrar rapidamente spans que falharam E foram lentos — esses são os candidatos ideais para investigação.

04 — Agregações e Métricas de Traces

O TraceQL suporta **funções de agregação** que transformam resultados de spans em métricas numéricas — úteis para dashboards no Grafana e análises de SLO/SLA:

Função	O que calcula	Exemplo
<code>count()</code>	Quantidade de spans	<code>{ status=error } count() > 3</code>
<code>avg(attr)</code>	Média de um atributo	<code>{ } avg(duration)</code>
<code>max(attr)</code>	Valor máximo	<code>{ } max(duration) > 5s</code>
<code>min(attr)</code>	Valor mínimo	<code>{ } min(span.db.rows)</code>
<code>sum(attr)</code>	Soma dos valores	<code>{ } sum(span.bytes_sent)</code>
<code>rate()</code>	Traces por segundo (no Grafana)	<code>{ status=error } rate()</code>

Exemplos Completos e Práticos

■ Todas as requisições lentas no serviço de pagamentos

```
{
resource.service.name = "pagamentos"
&& duration > 2s
}

# Retorna todos os traces do serviço pagamentos
# onde algum span levou mais de 2 segundos
```

■ Traces com erros HTTP 5xx em qualquer serviço

```
{
span.http.status_code >= 500
&& span.http.status_code < 600
}

# Útil para encontrar falhas independente do serviço
# Resultado: lista de traces com spans de erro servidor
```

■ Rastrear lentidão no banco de dados (queries > 1s)

```
{
span.db.system != nil
&& duration > 1s
}

# span.db.system != nil → é um span de banco de dados
# Identifica queries SQL/NoSQL que estão lentas
```

■ Traces onde a requisição passou por mais de 5 spans com erro

```
{ status = error } | count() >= 5

# Falhas em cascata – muitos microsserviços falhando
# Alto número de spans com erro = problema sistêmico
```

A Tríade da Observabilidade Bradesco

Ferramenta	Linguagem	Responde a...
------------	-----------	---------------

Grafana Loki	LogQL	O que aconteceu? (eventos e erros)
Prometheus	PromQL	Qual o estado atual? (métricas e alertas)
Grafana Tempo	TraceQL	Por onde passou? (jornada da requisição)

■ **Dica:** Para praticar TraceQL, acesse **play.grafana.org** → Explore → selecione a fonte **Tempo**. Você pode explorar traces reais e testar queries sem instalar nada.